

React.js Training Roadmap — Intern Program

A structured path from zero to production-ready React. Starts from HTML/CSS/JS fundamentals, then builds up to React.

Phase 0: Web & JavaScript Fundamentals

Don't skip this — shaky JS fundamentals is the #1 reason beginners struggle with React later. Give this phase real time.

0.1 HTML & CSS Basics

- Semantic HTML structure (headings, forms, lists, divs vs semantic tags)
- CSS box model, flexbox, basic responsive layout
- Building a couple of static pages by hand (no frameworks)

0.2 JavaScript Fundamentals

- Variables (`let` / `const`), data types, operators
- Functions, scope, `this`
- Arrays and objects, basic looping
- DOM manipulation basics (`document.querySelector` , event listeners) — helps them appreciate what React automates later

0.3 Modern JavaScript (ES6+)

- Arrow functions
- Destructuring, spread/rest operators
- Template literals
- Array methods: `map` , `filter` , `reduce` , `find`
- Modules (`import` / `export`)

0.4 Asynchronous JavaScript

- Callbacks → Promises → `async` / `await`

- `fetch` API — making a basic GET request to a public API (no auth needed yet)
- Understanding what an API response looks like (JSON)

0.5 Version Control Basics

- Git fundamentals: clone, commit, push, pull, branches
- Basic GitHub workflow (since they'll use this constantly in React projects)

Suggested checkpoint project: a small static page (e.g. a "profile card" or simple calculator) built with vanilla JS, no React — confirms fundamentals before moving on.

Phase 1: React Foundations

1.1 JavaScript Prerequisites Check

- ES6+ essentials: arrow functions, destructuring, spread/rest, template literals
- Array methods: map, filter, reduce
- Promises, `async/await`
- Modules (`import/export`)

1.2 React Core Concepts

- What is React, why component-based UI, Virtual DOM concept
- JSX syntax and rules
- Functional components
- Props (passing data down, prop types)
- Rendering lists and using `key` correctly
- Conditional rendering (ternary, `&&`, early returns)

1.3 Setup & Tooling

- Create React App vs Vite (recommend Vite)
- Project structure conventions
- React DevTools browser extension

Phase 2: State & Interactivity

2.1 State Management Basics

- `useState` — reading and updating state
- Functional updates (`setState(prev => ...)`)
- State immutability (why direct mutation breaks things)

2.2 Event Handling

- Synthetic events, event handlers
- Forms: controlled components (`value + onChange`)
- Form validation basics

2.3 Component Communication

- Lifting state up
- Passing callbacks as props (child → parent communication)
- Prop drilling — recognizing the pain point (sets up Context later)

2.4 Side Effects

- `useEffect` — dependency array behavior (no array, empty array, with values)
- Cleanup functions (avoiding memory leaks)
- Common use cases: data fetching, subscriptions, timers

Phase 3: Intermediate Patterns

3.1 More Hooks

- `useRef` — DOM access, persisting values without re-render
- `useMemo` — memoizing expensive calculations
- `useCallback` — memoizing functions
- `useContext` — avoiding prop drilling with Context API

3.2 Component Architecture

- Composition vs inheritance
- Container/presentational component pattern

- Fragments (`<>...</>`)
- Custom hooks — extracting reusable logic (e.g. `useFetch` , `useToggle`)

3.3 Styling Approaches

- CSS Modules
- Styled-components / Tailwind (pick one to standardize on)
- Conditional class names

3.4 Working with APIs

- Fetching data with `fetch` / `Axios` inside `useEffect`
- Loading and error states
- Connecting to their existing Flask/REST APIs from earlier training

Phase 4: Routing & App Structure

4.1 React Router

- Setting up routes, nested routes
- `Link` vs `Navigate` vs programmatic navigation
- Route params and query params
- Protected/private routes (auth-gated pages)

4.2 Project Structure at Scale

- Feature-based folder structure
- Reusable component libraries
- Environment variables (`.env`) for API URLs

Phase 5: State Management at Scale

5.1 Beyond Context

- When Context isn't enough (performance, complex state logic)
- `useReducer` for complex local state
- Intro to a state library: Redux Toolkit or Zustand (pick one)

5.2 Server State

- Difference between client state and server state
- React Query / TanStack Query — caching, refetching, invalidation
- Reducing manual `useEffect` + `useState` fetching boilerplate

Phase 6: Testing & Quality

6.1 Testing

- Jest basics
- React Testing Library — testing components by behavior, not implementation
- Writing tests for a form, a list, and an API-connected component

6.2 Code Quality

- ESLint + Prettier setup
- Component design principles (single responsibility, avoiding prop bloat)
- Performance basics: when to use `React.memo`, avoiding unnecessary re-renders

Phase 7: Production Readiness

7.1 Build & Deploy

- Production builds, environment configs
- Deploying to Vercel/Netlify or a container alongside their Flask backend
- Basic CI check (lint + test on push)

7.2 Advanced Topics (as time allows)

- Code splitting / lazy loading (`React.lazy`, `Suspense`)
- Error boundaries
- Accessibility basics (semantic HTML, ARIA where needed)

Capstone Project

A small full-stack app that forces them to combine everything:

- Multi-page app with React Router
- Auth flow (login/protected routes) hitting their Flask backend
- Form with validation, connected to a REST API
- List/detail views with loading and error states
- At least one custom hook and one piece of global state (Context or Redux/Zustand)
- Basic tests for 2–3 components

Suggested project ideas: a mini job board, a ticket/issue tracker, or an inventory management tool — all pair well with what they already built in Python/Flask.